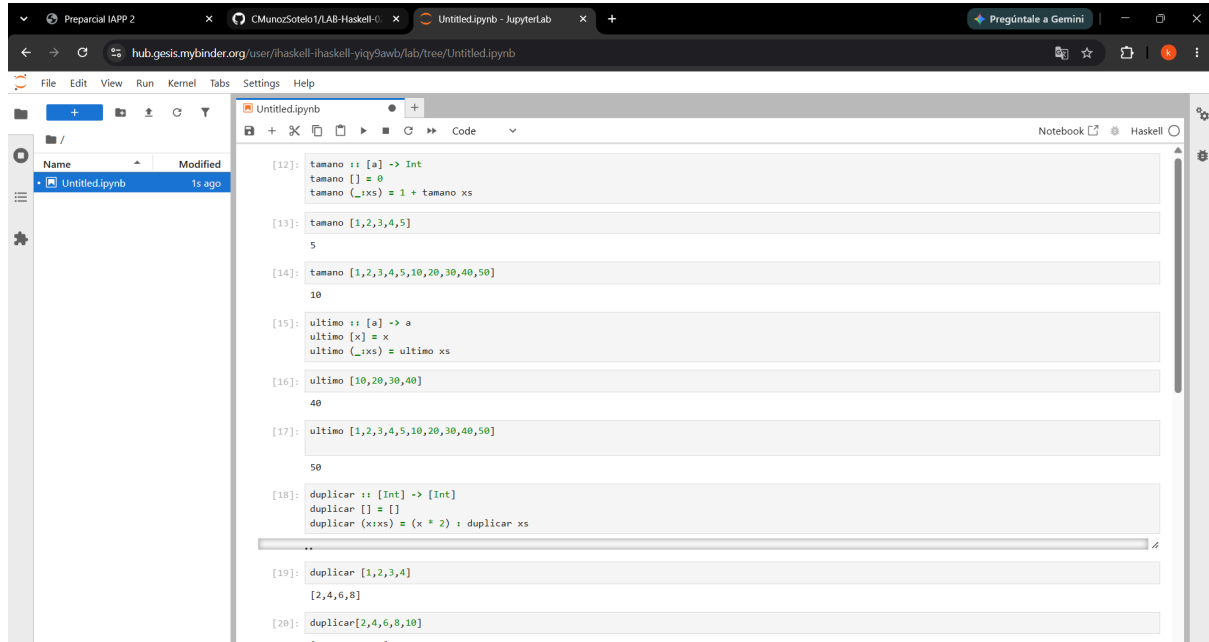


Camilo Andres Muñoz Sotelo

Punto 1

Definir las siguientes funciones utilizando recursión



```
[12]: tamaño :: [a] -> Int
      tamaño [] = 0
      tamaño (_:xs) = 1 + tamaño xs

[13]: tamaño [1,2,3,4,5]
      5

[14]: tamaño [1,2,3,4,5,10,20,30,40,50]
      10

[15]: ultimo :: [a] -> a
      ultimo [x] = x
      ultimo (_:xs) = ultimo xs

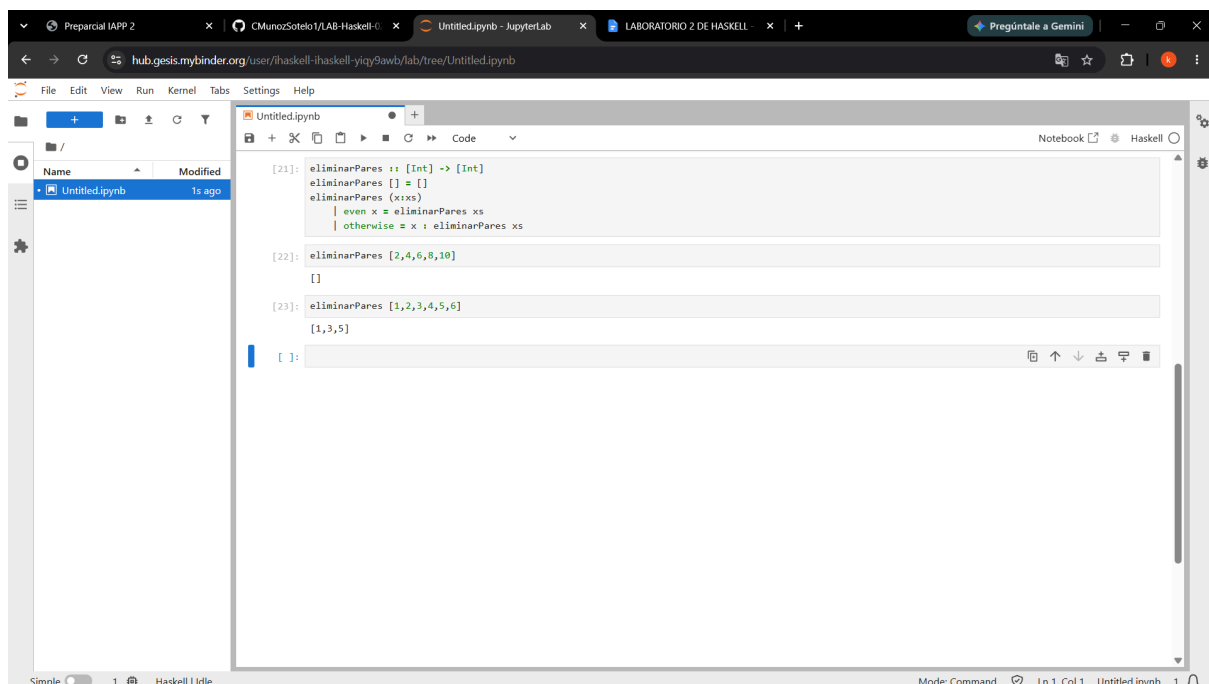
[16]: ultimo [10,20,30,40]
      40

[17]: ultimo [1,2,3,4,5,10,20,30,40,50]
      50

[18]: duplicar :: [Int] -> [Int]
      duplicar [] = []
      duplicar (x:xs) = (x * 2) : duplicar xs

[19]: duplicar [1,2,3,4]
      [2,4,6,8]

[20]: duplicar [2,4,6,8,10]
      [4,8,12,16,20]
```



```
[21]: eliminarPares :: [Int] -> [Int]
      eliminarPares [] = []
      eliminarPares (x:xs)
        | even x = eliminarPares xs
        | otherwise = x : eliminarPares xs

[22]: eliminarPares [2,4,6,8,10]
      []

[23]: eliminarPares [1,2,3,4,5,6]
      [1,3,5]

[ ]:
```

Define estas mismas funciones pero utilizando funciones de orden superior como map y filter

The screenshot shows a JupyterLab interface with a file browser on the left and a code editor on the right. The file browser shows a directory with two files: 'Untitled1.ipynb' and 'Untitled1.ipynb'. The code editor shows the following Haskell code:

```
[1]: duplicar2 :: [Int] -> [Int]
    duplicar2 = map (*2)

[2]: duplicar2 [1,2,3,4]
[2,4,6,8]

[3]: duplicar2 [10,20,30,40]
[20,40,60,80]

[4]: eliminarPares2 :: [Int] -> [Int]
    eliminarPares2 = filter odd

[5]: eliminarPares2 [1,2,3,4,5,6]
[1,3,5]

[6]: eliminarPares2 [10,20,30,40]
[]

[ ]:
```

Punto 2

1. Usa compresión de listas para
 - a. Obtener los cuadrados de los primeros 10 números.
 - b. Filtrar los divisibles por 3 entre 1 y 30.
 - c. Generar una lista de pares (x, y) donde $x < y$ y ambos estén en $[1..5]$.

The screenshot shows a JupyterLab interface with a file browser on the left and a code editor on the right. The file browser shows a directory with three files: 'Untitled1.ipynb', 'Untitled1.ipynb', and 'Untitled2.ipynb'. The code editor shows the following Haskell code:

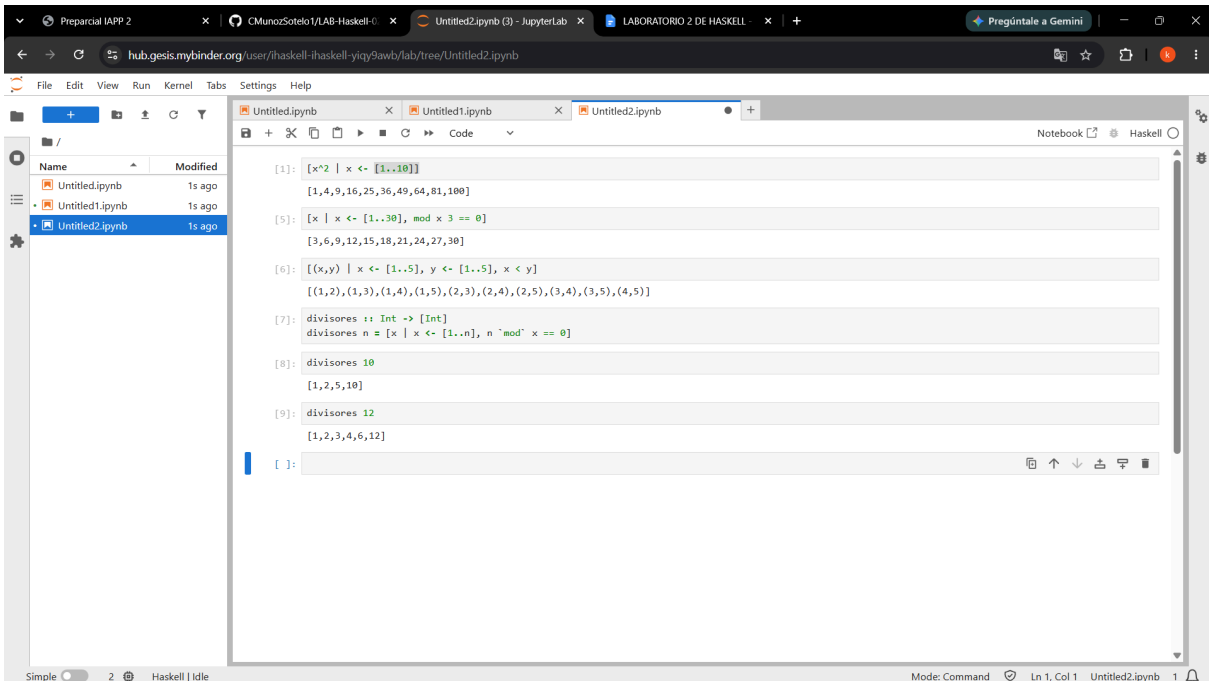
```
[1]: [x^2 | x <- [1..10]]
[1,4,9,16,25,36,49,64,81,100]

[5]: [x | x <- [1..30], mod x 3 == 0]
[3,6,9,12,15,18,21,24,27,30]

[6]: [(x,y) | x <- [1..5], y <- [1..5], x < y]
[(1,2),(1,3),(1,4),(1,5),(2,3),(2,4),(2,5),(3,4),(3,5),(4,5)]

[ ]:
```

2. Define una función que devuelva los divisores de un número (usando lista por comprensión):
divisores :: Int -> [Int]



The screenshot shows a JupyterLab notebook with the following code cells:

```
[1]: [x^2 | x <- [1..10]]
[1,4,9,16,25,36,49,64,81,100]

[5]: [x | x <- [1..30], mod x 3 == 0]
[3,6,9,12,15,18,21,24,27,30]

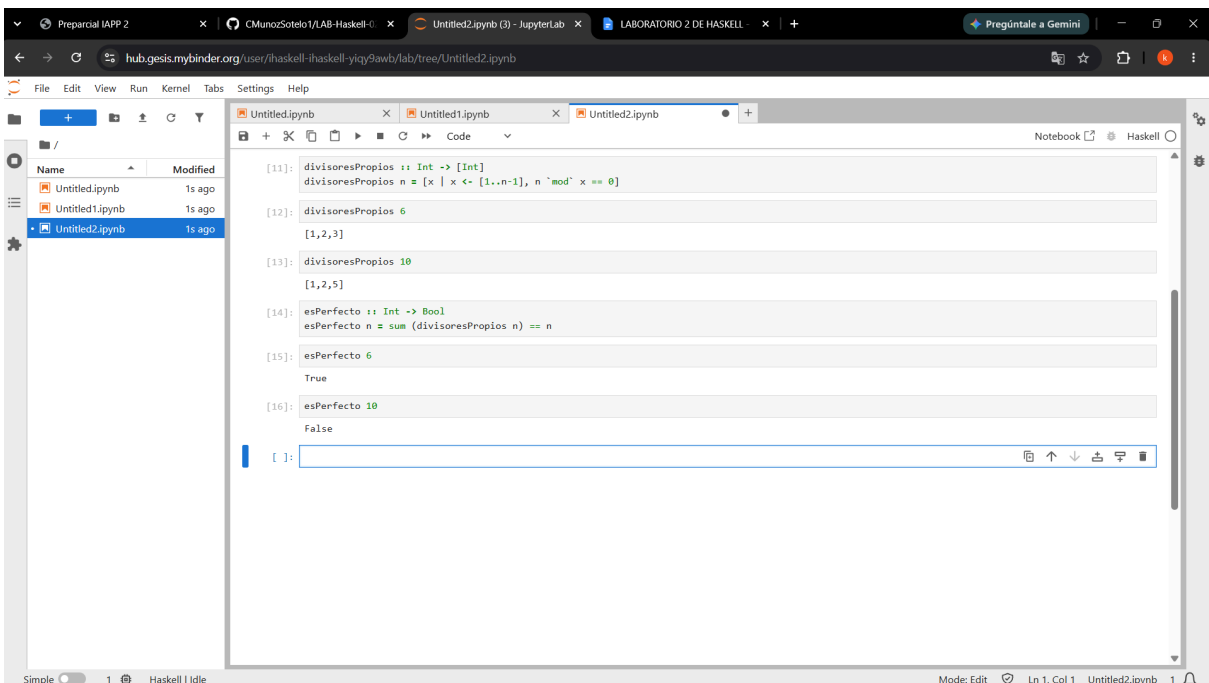
[6]: [(x,y) | x <- [1..5], y <- [1..5], x < y]
[(1,2),(1,3),(1,4),(1,5),(2,3),(2,4),(2,5),(3,4),(3,5),(4,5)]

[7]: divisores :: Int -> [Int]
divisores n = [x | x <- [1..n], n `mod` x == 0]

[8]: divisores 10
[1,2,5,10]

[9]: divisores 12
[1,2,3,4,6,12]
```

3. Define la función divisores propios, crea una función que determine si un número es perfecto



The screenshot shows a JupyterLab notebook with the following code cells:

```
[11]: divisoresPropios :: Int -> [Int]
divisoresPropios n = [x | x <- [1..n-1], n `mod` x == 0]

[12]: divisoresPropios 6
[1,2,3]

[13]: divisoresPropios 10
[1,2,5]

[14]: esPerfecto :: Int -> Bool
esPerfecto n = sum (divisoresPropios n) == n

[15]: esPerfecto 6
True

[16]: esPerfecto 10
False
```

Punto 3

1. Reescribe las siguientes funciones sin recursión explícita, usando funciones de orden superior.

The screenshot shows a JupyterLab interface with a file browser on the left and a code editor on the right. The file browser lists four files: Untitled1.ipynb, Untitled2.ipynb, and Untitled3.ipynb. The code editor displays the following Haskell code:

```
[1]: sumaLista :: [Int] -> Int
    sumaLista xs = sum xs

[2]: sumaLista [10,11,12]
33

[ ]:
```

The status bar at the bottom indicates "Simple", "1", "Haskell | Idle", and "Mode: Edit".

The screenshot shows a JupyterLab interface with a file browser on the left and a code editor on the right. The file browser lists one file: Untitled.ipynb. The code editor displays the following Haskell code:

```
[1]: mayoresQue10 :: [Int] -> Int
    mayoresQue10 xs = length (filter (>10) xs)

[2]: mayoresQue10 [5,12,8,20,15]
3

[3]: productolista :: [Int] -> Int
    productolista xs = product xs

[4]: productolista [1,2,3,4]
24

[5]: celsiusAFahrenheit :: [Double] -> [Double]
    celsiusAFahrenheit xs = map (\c -> c * 9 / 5 + 32) xs

[6]: celsiusAFahrenheit [0,10,20,30]
[32.0,50.0,68.0,86.0]

[7]: ajustarImpares :: [Int] -> [Int]
    ajustarImpares xs = map (\x -> if odd x then x + 5 else x) xs

[8]: ajustarImpares [1,2,3,4,5]
[6,2,8,4,10]

[ ]:
```

The status bar at the bottom indicates "Simple", "1", "Haskell | Idle", and "Mode: Command".

2. Define las siguientes funciones usando filter

The screenshot shows a JupyterLab notebook with the following code blocks:

```
[1]: nombresLargos :: [String] -> [String]
nombresLargos xs = filter (\nombre -> length nombre > 5) xs

[2]: nombresLargos ["Juan","Camilo","Pedro","Andres"]
["Camilo","Andres"]

[3]: soloNegativos :: [Int] -> [Int]
soloNegativos xs = filter (<0) xs

[4]: soloNegativos [5,-2,8,-10,3]
[-2,-10]

[5]: edadesUniversitarios :: [Int] -> [Int]
edadesUniversitarios xs = filter (\edad -> edad >= 18 && edad <= 25) xs

[6]: edadesUniversitarios [16,18,20,25,27,30]
[18,20,25]
```

3. Define las siguientes funciones utilizando filter y map

The screenshot shows a JupyterLab notebook with the following code blocks:

```
[1]: agregarIVA :: [Double] -> [Double]
agregarIVA xs =
  map (\x -> if x >= 1000 then x * 1.19 else x) xs

[2]: agregarIVA [500,1000,1500,800]
[500.0,1190.0,1785.0,800.0]

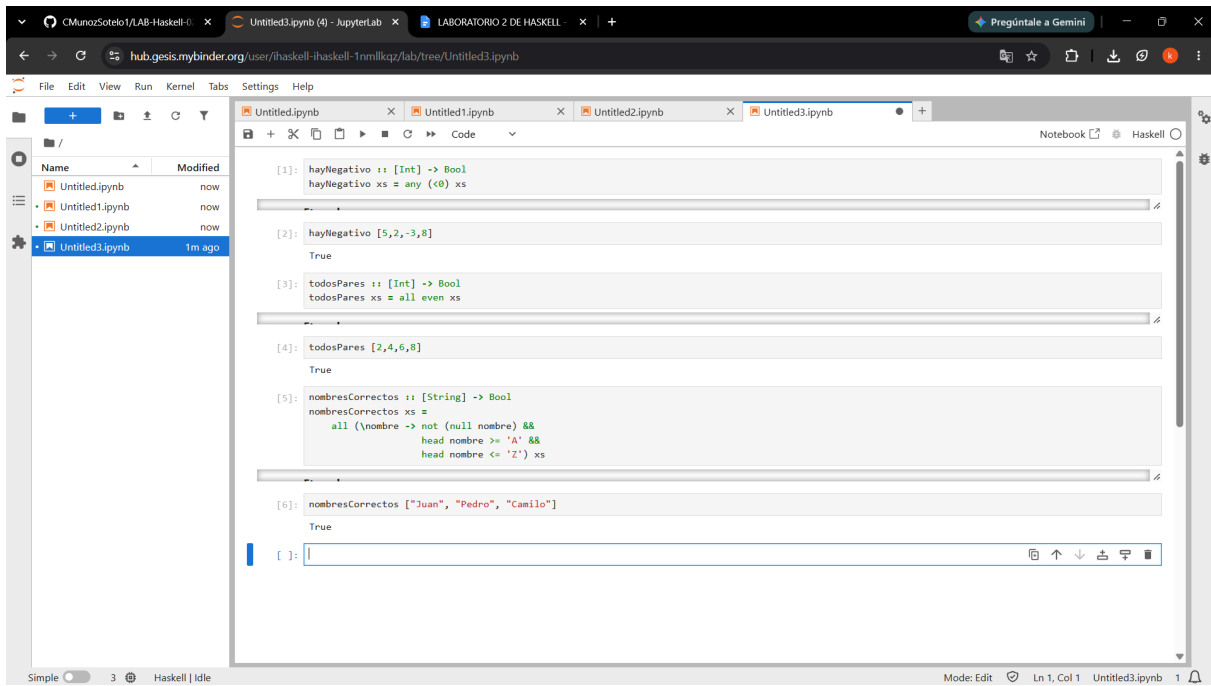
[3]: longitudesVocales :: [String] -> [Int]
longitudesVocales xs =
  map length (filter empiezaConVocal xs)
  where
    empiezaConVocal palabra =
      not (null palabra) &&
      head palabra `elem` "aeiouAEIOU"

[4]: longitudesVocales ["hola","arbol","casa","universidad","perro"]
[5,11]

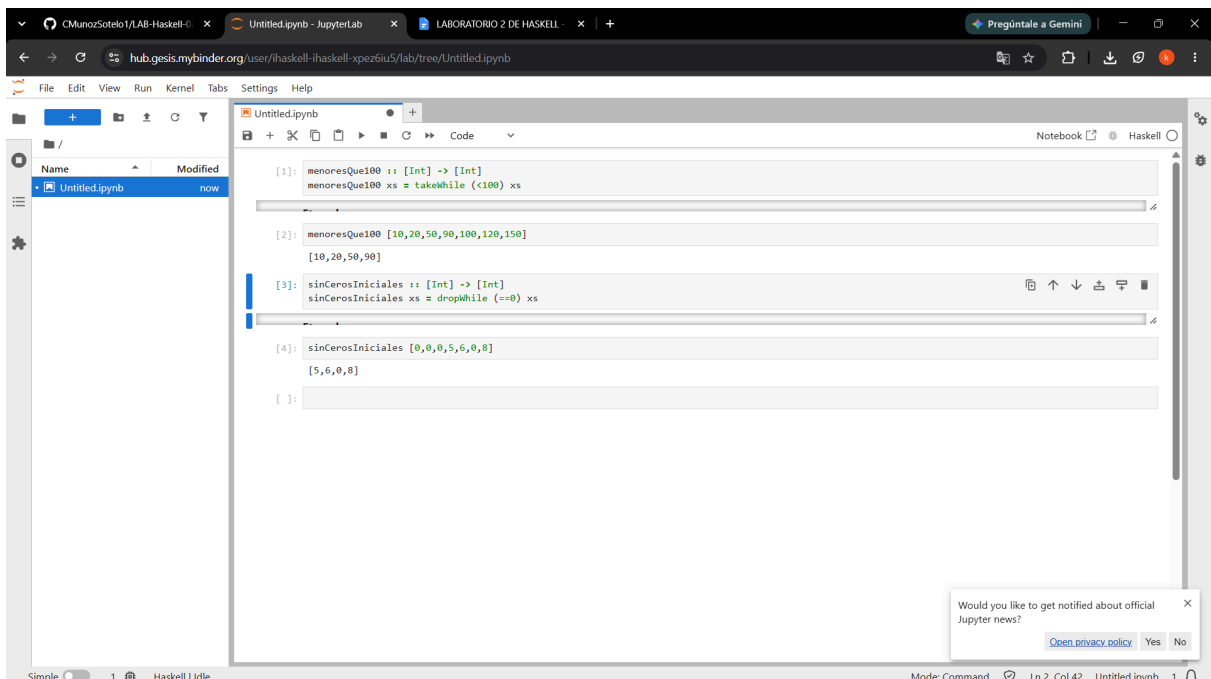
[5]: cuadradosPares :: [Int] -> [Int]
cuadradosPares xs = map (^2) (filter even xs)

[6]: cuadradosPares [1,2,3,4,5,6]
[4,16,36]
```

4. Define las siguientes funciones utilizando any o all



5. Define las siguientes funciones utilizando takeWhile o dropWhile



Punto 4

1. Define una función que recibe una función que transforme un dato (por ejemplo, conversión de grados Celsius a Fahrenheit), y lo aplica a una lista de datos.
2. Define una función que retorne un filtro personalizado
3. Crea una función que cree verificadores de rangos

The screenshot shows a JupyterLab notebook with the following code blocks:

```
[1]: procesarLecturas :: (Double -> Double) -> [Double] -> [Double]
    procesarLecturas funcion lista = map funcion lista

[2]: celsiusAFahrenheit :: Double -> Double
    celsiusAFahrenheit c = c * 9 / 5 + 32

[3]: procesarLecturas celsiusAFahrenheit [0,12,20]
[32.0,53.6,68.0]

[4]: crearFiltro :: (a -> Bool) -> ([a] -> [a])
    crearFiltro = filter

[5]: soloPares = crearFiltro even

[6]: soloPares [1..10]
[2,4,6,8,10]

[7]: enRango :: Int -> Int -> (Int -> Bool)
    enRango minimo maximo = \x -> x >= minimo && x <= maximo

[9]: entre5y10 = enRango 5 10

[10]: entre5y10 7
True
```

4. Genera una función que eleva una potencia dada

The screenshot shows a JupyterLab notebook with the following code blocks:

```
[1]: potenciador :: Int -> (Int -> Int)
    potenciador potencia = \x -> x ^ potencia

[2]: potencia5 = potenciador 5

[3]: potencia5 7
16807
```

Punto 5

Retrospectiva

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes?
(Horas/Hombre)

Camilo Andres Muñoz Sotelo, le dedique al laboratorio unas 5 - 6 horas

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?

3. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Mi mayor logro fue poder identificar la función del filter, map, any, all, takeWhile y dropWhile.

4. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

En el primer laboratorio no sabía donde podía ejecutar el código, en este pude solucionar esto y desarrollar el laboratorio ya con resultados.

5. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Solo soy yo, me comprometo a utilizar más ayuda del profesor para el desarrollo de los laboratorios.

6. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados

https://www.cs.us.es/~jalonso/cursos/i1m/doc/Funciones_basicas.html